

CASE STUDY REPORT

Debugging Pair Sum Code (Brute Force)

1. Title

Debugging and Optimization of Pair Sum Code Using Brute Force

2. Problem Description

The given Python program is designed to find two elements in an array whose sum is equal to a given target value. It should return the indices of the two elements. The original code performs unnecessary comparisons because both (i, j) and (j, i) may be checked. The loop structure can be improved to avoid duplicate comparisons.

3. Problem Statement

Given an array of integers and a target value, find a pair of different elements whose sum is equal to the target.

Input: arr = [2, 7, 11, 15], target = 9

Output: (0, 1)

Because $\text{arr}[0] + \text{arr}[1] = 2 + 7 = 9$.

4. Original Code

```
def pair_sum(arr, target):  
    for i in range(len(arr)):  
        for j in range(len(arr)):  
            if i != j and arr[i] + arr[j] == target:  
                return (i, j)  
    return None
```

5. Logical Errors and Issues

1. Duplicate Pair Checking: The inner loop starts from 0, so the same pair can be checked twice, such as (0,1) and (1,0).
2. Unnecessary $i \neq j$ Condition: This condition is required in the original program, but can be avoided by starting j from $i + 1$.
3. Return Condition: The function should return the pair immediately when a valid pair is found. If no pair exists, return None should execute only after all possible pairs have been checked.

6. Debugging Step-by-Step

For `arr = [2, 7, 11, 15]` and `target = 9`:

Step 1: `i = 0` and `arr[0] = 2`.

Step 2: `j = 1` and `arr[1] = 7`.

Step 3: $2 + 7 = 9$, which matches the target.

Step 4: Return `(0, 1)`.

Therefore, the valid pair is found successfully.

7. Corrected Brute-Force Algorithm

Change the inner loop to `range(i + 1, len(arr))`. This checks every pair only once and automatically prevents `i` from being equal to `j`.

8. Corrected Program

```
def pair_sum(arr, target):  
    for i in range(len(arr)):  
        for j in range(i + 1, len(arr)):  
            if arr[i] + arr[j] == target:  
                return (i, j)  
  
    return None
```

Test case

```
arr = [2, 7, 11, 15]
```

```
target = 9
```

```
result = pair_sum(arr, target)
```

```
print("Pair indices:", result)
```

9. Output

Pair indices: `(0, 1)`

10. Optimized Solution

```
def pair_sum(arr, target):  
    seen = {}  
  
    for i, value in enumerate(arr):  
        required = target - value
```

```
    if required in seen:
        return (seen[required], i)
```

```
    seen[value] = i
```

```
    return None
```

```
# Test case
```

```
arr = [2, 7, 11, 15]
```

```
target = 9
```

```
print("Pair indices:", pair_sum(arr, target))
```

11. Output

Pair indices: (0, 1)

12. Complexity Analysis

Brute-Force Method:

- Time Complexity: $O(n^2)$
- Space Complexity: $O(1)$

The program compares possible pairs of elements.

Optimized Method:

- Time Complexity: $O(n)$
- Space Complexity: $O(n)$

The dictionary stores previously visited elements and provides fast lookup.

13. Advantages of the Optimized Method

- Faster for large arrays.
- Avoids repeated comparisons.
- Uses a dictionary for quick lookup.
- Reduces time complexity from $O(n^2)$ to $O(n)$.

14. Limitations

- Requires additional memory for the dictionary.
- The brute-force method is simpler for small arrays.
- The optimized method is slightly more complex to understand.

15. Conclusion

The original pair-sum program can be improved by changing the loop structure so that each pair is checked only once. The corrected brute-force solution has $O(n^2)$ time complexity. For large arrays, the dictionary-based solution is more efficient because it reduces the time complexity to $O(n)$, although it requires $O(n)$ additional space.